

Álgebra Linear, Vetores e Geometria Analítica

Entrega 2

Grupo: Enrolados

Guilherme Barioni
Iury Xavier da Silva Mangueira
Lilian Mercedes Paye Conde
Marcus Miranda Duque

Definição da matriz de transformação

Para esta entrega, decidimos utilizar a matriz de cisalhamento. Ela consiste em uma transformação linear que permite deformar a imagem de maneira controlada, deslizando suas camadas paralelas a um eixo (x ou y) sem que haja alteração no seu volume ou área total.

Nossa aplicação foca no cisalhamento horizontal, cuja transformação é definida por: $Cx(x, y) = (x + ky, y)$. A matriz base para esta transformação horizontal é definida matematicamente como:

$$\begin{bmatrix} 1 & k \\ 0 & 1 \end{bmatrix}$$

Nesta formatação, o eixo y se mantém fixo, enquanto o eixo x é deslocado horizontalmente com base no valor atual de y. A variável k atua como o fator de cisalhamento (intensidade da deformação).

Também é possível aplicar um cisalhamento de forma vertical, onde a fórmula se inverte para $Cx(x, y) = (x, y + kx)$. Neste caso alternativo, o eixo x seria mantido fixo e o deslocamento aconteceria no eixo y em proporção ao valor de x e de k. A matriz base para esta transformação vertical é definida matematicamente com:

$$\begin{bmatrix} 1 & 0 \\ k & 1 \end{bmatrix}$$

Aplicação do Cisalhamento às Coordenadas Espaciais

Para aplicar a transformação linear na prática, optamos por utilizar a matriz gerada da imagem da primeira entrega. O processamento foi construído em um script na linguagem Python, fazendo uso primário das bibliotecas numpy e PIL (Pillow).

Carregamos os dados da matriz e lendo as linhas e colunas da tabela, isolamos os componentes R, G e B de cada coordenadas espacial original em uma variável de pixels e armazenamos suas dimensões (largura e altura) em variáveis distintas. Então, declaramos explicitamente o nosso fator de cisalhamento com $k = 0.5$, alimentando a nossa matriz de transformação com este peso.

Para evitar que a deformação fizesse a imagem sair do quadro e ficasse cortada, nós calculamos uma "nova largura" por meio da lógica associada à própria transformação de cisalhamento (**$\text{largura} + (\text{altura} * k)$**).

A partir destas novas dimensões acomodadas, criamos um fundo preto (nova imagem) que receberá os pixels e iteramos sobre a matriz da imagem com cada loop feito em uma coluna da imagem, realizamos um loop na linha correspondente.

Em cada coordenada iterada, criamos um vetor espacial original $v = \text{np.array}([x, y])$ e o multiplicamos matricialmente pela nossa matriz de transformação através da função **`matriz_transformacao.dot(v)`**. Os valores de RGB do pixel correspondente são então transferidos com base nas novas coordenadas `novo_x` e `novo_y` calculadas, gerando a nova configuração espacial final, os dados ficam disponíveis em uma imagem PNG e tabela csv.

```
import numpy as np
from PIL import Image
import csv

print("Lendo os dados do arquivo CSV original...")

# 1. Lendo os dados espaciais e de cor do CSV original
with open("pixels.csv", "r") as f:
    reader = list(csv.reader(f))

# Separa o cabeçalho dos dados
header = reader[0]
data = reader[1:]

height = len(data)
width = len(header) // 3 # Cada pixel ocupa 3 colunas (R, G, B)

# 2. Definição explícita da matriz de transformação (Cisalhamento)
k = 0.5
matriz_transformacao = np.array([
    [1, k],
    [0, 1]
])

# Calcula as novas dimensões para evitar cortes
nova_largura = int(width + (height * k))
nova_altura = height
```

```

# 3. Preparando as estruturas para os dois outputs (Imagem e CSV)

# A) Cria a nova imagem vazia (fundo preto)
nova_img = Image.new( mode: "RGB", size: (nova_largura, nova_altura), color: "black")
novos_pixels = nova_img.load()

# B) Cria a nova matriz de dados para o CSV (preenchida com zeros/preto)
# Uma lista de listas, onde cada linha tem (nova_largura * 3) colunas
nova_matriz_csv = [[0 for _ in range(nova_largura * 3)] for _ in range(nova_altura)]

print("Aplicando a transformação linear...")

# 4. Aplicação da matriz às coordenadas
for y, row in enumerate(data):
    for x in range(width):
        # Extrai os valores RGB do pixel original a partir do CSV
        r = int(row[x * 3])
        g = int(row[x * 3 + 1])
        b = int(row[x * 3 + 2])

        # Vetor de coordenadas espaciais originais
        v = np.array([x, y])

        # Multiplicação de matriz: v_novo = Matriz * v
        v_novo = matriz_transformacao.dot(v)

        novo_x = int(v_novo[0])
        novo_y = int(v_novo[1])

```

```

        # Verifica os limites para não dar erro de "out of bounds"
        if 0 <= novo_x < nova_largura and 0 <= novo_y < nova_altura:
            # 1° Output: Atualiza o pixel na nova imagem
            novos_pixels[novo_x, novo_y] = (r, g, b)

            # 2° Output: Atualiza os valores R, G e B na nova matriz CSV
            # Como cada pixel ocupa 3 posições, multiplicamos o x por 3
            nova_matriz_csv[novo_y][novo_x * 3] = r
            nova_matriz_csv[novo_y][novo_x * 3 + 1] = g
            nova_matriz_csv[novo_y][novo_x * 3 + 2] = b

# 5. Salvando os resultados

# Salva a imagem visual
nova_img.save("imagem_transformada.png")

# Salva a nova matriz no arquivo CSV
print("Gerando o novo arquivo CSV...")
with open("pixels_transformados.csv", "w", newline="") as f:
    writer = csv.writer(f)

    # Cria o novo cabeçalho (agora indo até a nova_largura)
    novo_header = []
    for x in range(1, nova_largura + 1):
        novo_header.extend([f"r{x}", f"g{x}", f"b{x}"])
    writer.writerow(novo_header)

```

Comparação entre imagem original e imagem deformada

Original



Imagem transformada



Conforme visualizado, a imagem deformada apresenta uma inclinação perceptível de suas colunas em direção à direita. Os espaços vazios gerados pelo deslocamento dos pixels e o

alongamento da moldura foram compensados corretamente pelo fundo preto calculado previamente.

Análise Conceitual e Efeitos Geométricos

O cisalhamento altera fundamentalmente a "forma da figura", convertendo o perímetro retangular original da fotografia em um paralelogramo inclinado.

Ao analisar como as alterações na matriz impactam o espaço:

- Como a matriz utilizada opera somando ky à coordenada x original, os pixels situados na linha de base ($y = 0$) da imagem não sofrem nenhum tipo de translação.
- À medida que as coordenadas y aumentam (ao subir pelas linhas da imagem), o deslocamento sobre o eixo horizontal x torna-se cada vez mais forte, causando o efeito visual de "baralho sendo empurrado" de forma contínua.
- O aumento da variável da matriz (o fator k) resultaria num tombo ainda maior da imagem, exigindo ainda mais lona lateral (fundo preto) para acomodá-la.

Preservação de Dimensão e Colapso Dimensional

O cisalhamento é um excelente exemplo de uma transformação que preserva perfeitamente a dimensão do espaço. O determinante da matriz de cisalhamento implementada:

$$\begin{bmatrix} 1 & k \\ 0 & 1 \end{bmatrix}$$

É igual a 1 ($(1 * 1) - (k * 0) = 1$). O fato do determinante ser 1 indica que, embora a imagem incline e sua forma periférica mude para um paralelogramo, a área geométrica total da imagem original se mantém constante, preservando a integralidade de seus dados dentro do espaço bidimensional original (R^2).

Em contrapartida, transformações sofrem **colapso dimensional** quando o determinante de suas matrizes resulta em zero. Por exemplo, caso alterássemos o código para utilizar uma matriz de projeção no eixo horizontal, como:

$$\begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}$$

Os novos valores espaciais de y resultariam todos em 0. Isso achataria e colapsaria inteiramente a nossa imagem de duas dimensões (R^2) para apenas uma única linha unidimensional (R^1), causando perda irreversível das informações visuais originais.

Conclusão

O desenvolvimento desta entrega evidenciou, de maneira tátil e observável, que as transformações lineares ultrapassam os limites de operações puramente teóricas ou abstratas, consistindo nos mecanismos fundamentais que regem a computação gráfica e o processamento de imagens.

Ficou provado que a escolha de uma matriz determina explicitamente o comportamento geométrico da imagem original. Para ilustrar a relevância desse conceito, o próprio cisalhamento possui diversas utilizações diretas e cotidianas na computação gráfica aplicada. Alguns exemplos práticos notáveis incluem:

- **Geração de Tipografia (Itálico):** A inclinação de fontes normais para a criação de textos em estilo *itálico* ou oblíquo é, na prática, uma matriz de cisalhamento aplicada aos contornos (vetores) das letras.
- **Projeção de Sombras Artificiais:** A simulação de sombras dinâmicas frequentemente utiliza o cisalhamento. A silhueta do objeto principal é "cisalhada" contra o chão para replicar o ângulo em que a luz o atinge, criando uma sombra inclinada que acompanha a forma do objeto.
- **Perspectiva Isométrica e "Falso 3D" (2.5D):** Em jogos eletrônicos 2D e plantas arquitetônicas, o cisalhamento é usado para distorcer "sprites" (imagens bidimensionais) e cenários, criando a ilusão de profundidade e perspectiva espacial sem o custo computacional de renderizar modelos reais em três dimensões.
- **Efeitos de Velocidade:** Deformar temporariamente uma imagem através do cisalhamento horizontal é uma técnica clássica de animação para simular o efeito visual de alta velocidade de um corpo em movimento.

Por fim, ao analisar esses impactos tanto no espaço matricial do código quanto na aplicação visual final, consolida a Álgebra Linear não apenas como uma disciplina matemática, mas como a ferramenta central para a modelagem e transformação de dados em toda a computação visual.